Review Report: Lean Formalization of Van Horn's Uniqueness Theorem

Project: plausibility — Lean 4 (v4.33.1) + Mathlib (pinned v4.33.1) Paper: Kevin S. Van Horn, *From propositional logic to plausible reasoning: A uniqueness theorem*, IJAR 88 (2017) 309–332 Status: COMPLETE — semantic theorem, syntactic Lemma-6 bridge (R1+R2 -> count dependence), R3/R4 transfer (toSystem), and the final vanHorn_calibration theorem all machine-verified; zero sorry, axioms [propext, Classical.choice, Quot.sound] only Verification: lake build green · #print axioms on every theorem -> [propext, Classical.choice, Quot.sound] only · zero sorry/admit (grep + no sorryAx in axiom audit)

---

1. Executive summary

The paper's core argument — *plausibility values are forced to be rational probabilities* — is machine-checked end-to-end, from the raw Requirements R1–R4: the semantic finite-event layer (Theorems 14 and 16, Corollaries 8, 12, 15), the syntactic Lemma 6 / Corollary 8 from raw R1+R2 (Reduction.lean), the R3/R4 transfer (Bridge.lean: value_scale, value_mono_canonical, toSystem with all three structural fields), and the final calibration theorem vanHorn_calibration: lp.value A X = Y_1 (toSystem lp) (probOf (event of A-worlds within X-worlds)). A consistency witness (Theorem 16), a counterexample showing R4 is not derivable from the rest, and a nonuniform-probability example round out the verification.

The one-sentence content of the verified theorem:

> R1+R2 (as semantic change-of-variables) erase representational differences, R3 erases common multiplicity, R4 preserves strict inclusion; after those eliminations the only information left is the rational ratio of favorable to possible cases.

2. What is proved, and where

Semantic layer (complete)

Paper	Lean declaration	File
Definition 5 (change of variables)	EventEquiv	Basic.lean
Semantic R1+R2 / R3 / R4	PlausibilitySystem fields equiv_invariance, irrelevant_product, strict_mono	Basic.lean
Lemma 6 / Corollary 8 (canonical reduction)	eventEquiv_canonical, score_eq_canonical	Canonical.lean
Lemma 10 (scale invariance)	$upsilon_2$_scale, plus transport $upsilon_2$_denom_eq	ScaleInvariant.lean
Y_1 well-defined	$upsilon_2$_eq_of_rat_eq	RationalRepresentation.lean
Lemma 13 (strict monotone)	$upsilon_1$_strictMono	RationalRepresentation.lean
Corollary 12 (iff-forms)	score_eq_iff_probability_eq, score_lt_iff_probability_lt	ProbabilityTheorem.lean
Theorem 14 (representation + order iso)	score_eq_$upsilon_1$, plausibilityOrderIso : Rat01 ≃o ↥sys.PlausibilityRange	ProbabilityTheorem.lean

Paper	Lean declaration	File
Theorem 16 (consistency)	countSystem (counting measure satisfies all fields)	ProbabilityTheorem.lean
Bridge to propositional problems	eventOf, score_eventOf	ProbabilityTheorem.lean

Syntactic layer

Content	Lean declaration	File
Atom lifting / support	Formula.mapLift, Formula.support, eval_mapLift	VanHorn/Substitution.lean
R2 semantic content: definitions preserve models	defFormula, defineVal, r2Equiv, card_models_def_and	VanHorn/Substitution.lean
R3 semantic content: vocabularies split as products	sumEquiv, card_models_sum	VanHorn/Substitution.lean
Lemma 6's Z_i (diagram isolating one assignment)	diagramLit, diagram, eval_diagram, models_diagram	VanHorn/Substitution.lean
Syntactic Requirements R1–R4	LogicalPlausibility structure with fields r1 r2 r3 r4	VanHorn/Requirements.lean
Corollary 15: complement & product rules	probability_complement, probability_conjunction (+ card_models_neg_and, models_and_and_assoc)	VanHorn/ProbabilityLaws.lean
Unsatisfiable-premise convention (separate)	extendedProbability, extended-Probability_eq_of_satisfiable	VanHorn/ProbabilityLaws.lean
R4 necessity counterexample	threeSystem with equiv_invariance, irrelevant_product, not_strictMono	VanHorn/Counterexamples.lean
Nonuniform example	biased_coin_heads : countSystem.score (8-of-10 event) = 4/5	VanHorn/Counterexamples.lean
Syntax + models + sanity	Formula, models, logicalProbability + 4 exhaustive checks	PropLogic/
Syntactic Requirements over Atom := ℕ	LogicalPlausibility (eval-based Eqv/EqvAt/Entails/Sat, support-fresh R2/R3)	VanHorn/Requirements.lean
Iterated R2 (add/remove lists of definitions)	r2_many, r2_many_zip	VanHorn/Reduction.lean
Step 2: query becomes favorable-symbol disjunction	step2_eqvAt	VanHorn/Reduction.lean
Step 3: premise becomes exactly-one	eval_sBlock_eq, step3_premise_eqv	VanHorn/Reduction.lean
Lemma 6 (four-step reduction)	lemma6_canonical	VanHorn/Reduction.lean

Content	Lean declaration	File
Corollary 8 (syntactic)	lemma6_counts, value_eq_of_counts_eq	VanHorn/Reduction.lean
Canonic formula machinery + modelsOn	bigOr/exactlyOne/allNeg, modelsOn, card_modelsOn_subset	VanHorn/Canonic.lean

Bridge layer (Bridge.lean, closing the loop)

Content	Lean declaration
exactlyOne has exactly n models	card_modelsOn_exactlyOne
disjoint vocabularies multiply	card_modelsOn_split
favorable count via bigOr filter	card_modelsOn_bigOr_and_exactlyOne
Syntactic Lemma 10 (scale, via R3)	value_scale
Canonical strict monotonicity (via R4)	value_mono_canonical
The instance: Requirements $\to$ system	toSystem, canonicalValue
Final calibration theorem	vanHorn_calibration

3. Architecture

Plausibility.lean (root: imports everything)
Plausibility/Basic.lean EventEquiv, PlausibilitySystem (PartialOrder P)
Plausibility/Canonical.lean canonicalEvent, Corollary 8
Plausibility/ScaleInvariant.lean Lemma 10
Plausibility/RationalRepresentation.lean Rat01 = $\{q : \mathbb{Q} // 0 \le q \wedge q \le 1\}$, upsilon$_1$, Lemma 13
Plausibility/ProbabilityTheorem.lean Theorem 14 + 16 + iff-forms + eventOf bridge
Plausibility/PropLogic/Formula.lean syntax, eval
Plausibility/PropLogic/Semantics.lean models, Satisfiable, logicalProbability
Plausibility/VanHorn/Substitution.lean mapLift, r2Equiv, sumEquiv, diagram
Plausibility/VanHorn/Requirements.lean LogicalPlausibility (syntactic R1–R4)
Plausibility/VanHorn/Canonic.lean bigOr/exactlyOne, modelsOn, model counts
Plausibility/VanHorn/Reduction.lean Lemma 6 + Corollary 8 from raw R1+R2
Plausibility/VanHorn/ProbabilityLaws.lean Corollary 15, extendedProbability
Plausibility/VanHorn/Counterexamples.lean threeSystem (R4 violation), biased coin
Plausibility/VanHorn/Bridge.lean R3/R4 transfer, toSystem, vanHorn_calibration

Layering is the paper's own:

Propositional problem (A | X) [Requirements.lean — syntactic R1–R4]
↓ R1+R2, Lemma 6 ← PROVED (Reduction.lean: lemma6_canonical,
↓ lemma6_counts, value_eq_of_counts_eq)
Finite event, m favorable of n worlds [Basic, Canonical]
↓ R3 [ScaleInvariant]
Only the ratio m/n matters [RationalRepresentation]
↓ R4
Plausibility order $\cong \mathbb{Q} \cap [0,1]$ [ProbabilityTheorem]

↑ R3/R4 transfer (Bridge.lean: value_scale, value_mono_canonical,
↑ toSystem, vanHorn_calibration — all proved)

4. Design decisions a reviewer should know

1. Range, not ambient codomain. plausibilityOrderIso targets ↥sys.PlausibilityRange — the *achieved* values, ranging over world spaces of all finite sizes — never the ambient P (which may contain junk; e.g. P = ℚ × Bool with unused (q, true) values).
2. PartialOrder P, linearity derived. The ambient order is only partial; that the achieved values form a chain is a *consequence* (via upsilon_1_strictMono), matching the paper's stronger claim.
3. Rational [0,1], not real. Rat01 := {q : ℚ // 0 ≤ q ∧ q ≤ 1} (an abbrev, so instance resolution sees the subtype). No measure theory anywhere.
4. Universe polymorphism via ULift. PlausibilitySystem scores world spaces in Type u; canonical events live in ULift (Fin n) so they sit in the same universe. No fixed finite atom type is used for the range — the red-alert failure mode ("fixed α has only finitely many achieved values") is structurally avoided.
5. Dependent-transport style. Where Lean's dependent types block rewriting (Fin (a+b) vs Fin n), the proofs transport through explicit EventEquives (finCongr, Equiv.ulift) and defeq-tolerant calc chains rather than rw.
6. Layer labeling — closed. equiv_invariance is stated as a field of PlausibilitySystem (the semantic consequence of R1+R2, paper's Corollary 9); the bridge (Reduction.lean + Bridge.lean) now derives it from the raw syntactic R1/R2 via toSystem and vanHorn_calibration, so the semantic layer's assumption is discharged for every LogicalPlausibility.

5. The bridge — DONE

Goal (achieved): from lp : LogicalPlausibility P (syntactic R1–R4 fields), construct sys : PlausibilitySystem P and prove the calibration theorem.

All of the following are proved in Plausibility/VanHorn/Bridge.lean, all compiling, all axiom-clean:

1. Model counting on modelsOn — card_modelsOn_exactlyOne (the exactly-one formula has exactly n models), card_modelsOn_split (disjoint vocabularies multiply: #(φ∧ψ) = #φ · #ψ), card_modelsOn_bigOr_and_exactlyOne (favorable count: #(bigOr lt ∧ exactlyOne l) = #lt).
2. Syntactic scale invariance (Lemma 10 at the formula level) — value_scale: value (Q_m t) (One_n t) = value (Q_{k·m} t') (One_{k·n} t'), proved via R3 with shared fresh atoms and lemma6_counts on both sides with a common base.
3. Canonical strict monotonicity — value_mono_canonical: m < m′ ≤ n implies strict inequality of the canonical values, via R4 with a concrete witness valuation.
4. The instance — toSystem : LogicalPlausibility P → PlausibilitySystem P with score A := canonicalValue lp A.card (Fintype.card _); equiv_invariance (card-congruence), irrelevant_product (via value_scale), strict_mono (via value_mono_canonical) all proved.
5. The final theorem — vanHorn_calibration: lp.value A X = upsilon_1 (toSystem lp) (probOf (event of A-worlds within X-worlds)). Proof route: the attach-filter's card is the favorable count (Finset.sum_attach); lemma6_counts reduces value A X to the canonical problem at a fresh base b; value_scale with k = 1 transports the base to 1 (where canonicalValue lives); $\text{score\_eq\_upsilon}_1$ identifies the score with Y_1 (probOf event).

With this, Van Horn's Theorem 14 applies verbatim to the raw Requirements: R1–R4 force plausibility

to be the rational probability #(A∧X) / #X, and the achieved plausibility values are order-isomorphic to $\mathbb{Q} \cap [0,1]$.

6. Where to pivot / restructure if you change course

- NNRat refactor: Rat01 could become {q : $\mathbb{Q}_{\geq 0}$ // q ≤ 1} and logicalProbability NNRat.divNat. This deletes the num ≥ 0/natAbs plumbing in RationalRepresentation.lean (lemmas num_natAbs_le_den, the Int.natAbs_of_nonneg dances). Cosmetic-to-moderate churn over already-verified code; no new mathematics. Do it only if you'll extend that file.
- Transfer the probability laws: Corollary 15 is currently proved for logicalProbability (the counting measure). A one-line corollary via score_eq_upsilon$_1$ would state the laws for *every* plausibility system's calibration — high presentation value, trivial effort.
- modelsOn support invariance: prove #(models over larger atom set) scales both counts by 2^k so ratios are invariant (use sumEquiv with β := Bool). Formalizes "counting over a bigger language doesn't change the ratio". Evening-sized.

7. Possible error points — what a reviewer should check hardest

1. equiv_invariance provenance (Basic.lean): it *assumes* the semantic consequence of R1+R2 — but that consequence is now derived: Reduction.lean proves Lemma 6 / Corollary 8 from raw syntactic R1+R2, and Bridge.lean constructs the full PlausibilitySystem instance and the calibration theorem. The loop is closed; "R1–R4 ⍰ probability" is a theorem of this development, not a caveat.
2. strict_mono field semantics (Basic.lean): stated as A ⊂ B -> score A < score B. In this Mathlib, A ⊂ B decomposes as A ⊆ B ∧ ¬(B ⊆ A) (note: Lean's Finset ssubset is *not* ⊆ ∧ ≠ here — see upsilon$_1$_strictMono's proof, which case-analyzes hEq : B ⊆ A). Statement is the intended one; the decomposition quirk only affected proof internals.
3. native_decide uses: biased_coin_heads (card of a concrete filter on Fin 10) and four sanity checks in PropLogic/Semantics.lean. native_decide trusts the compiler for decidable ℕ/Bool facts — standard practice, but replaceable by decide if you want kernel-only evidence (slower to compile).
4. diagram depends on Finset.univ.toList ordering (Substitution.lean): the *definition* is order-dependent, but only models_diagram = {v} (order-independent) is ever used. If you later need syntactic identity of diagrams, revisit.
5. DecidableEq assumptions: models requires [DecidableEq α]; all transfer lemmas carry it. A classical-variant API would drop it at the cost of computability; current choice keeps native_decide checks possible.
6. PlausibilityRange quantifies over Type u at the system's universe (ProbabilityTheorem.lean): a system fixed at universe u never scores spaces in Type v for v ≠ u. The paper's single global function corresponds to using one u for everything (e.g. u := 1, since all finite types lift). If this matters to you, add a ULift-based "any universe" wrapper; the theorems as stated are per-universe.
7. countSystem at P := ℚ proves consistency of the three structural fields (semantic R1+R2, R3, R4). The syntactic consistency (paper's Theorem 16, for LogicalPlausibility) follows by composing the counting construction with the bridge once a syntactic count instance is written; the semantic half is what the uniqueness theorem needs.
8. Subagent-authored files (ProbabilityLaws.lean, Counterexamples.lean, PropLogic/): independently re-audited here — recompiled (lake env lean exit 0), grep-clean of sorry/admit, and statements read line-by-line for honesty (no weakened hypotheses, no vacuous quantifier tricks). threeSystem.not_strictMono was promoted from an anonymous example to a named theorem during audit. Residual risk: low but non-zero; the two files are short and readable.

8. How to rebuild and verify

```
export PATH="$HOME/.elan/bin:$PATH"
lake build                          # full build (green as of this report)
lake env lean Plausibility/VanHorn/Bridge.lean      # the bridge (final theorem)
lake env lean Plausibility/ProbabilityTheorem.lean  # single-file type-check example
grep -R -n -E '\b(sorry|admit)\b' . --include='*.lean' --exclude-dir=.lake  # must print nothing
```

Axiom audit (all results currently): #print axioms <name> -> [propext, Classical.choice, Quot.sound] — the three standard Lean axioms; no sorryAx, no custom axioms; R1–R4 enter only as explicit structure fields.

9. Inventory at a glance

- 15 Lean files, ~3,800 lines of proof source.
- 180+ named theorems/defs across semantic and syntactic layers; every one axiom-clean (propext, Classical.choice, Quot.sound only; zero sorry).
- Commits: d5a783f (semantic layer), 90e73ff (bridge infrastructure + laws + counterexamples), 2fb2af4 (Lemma 6 core), 7abf2c4 (syntactic Corollary 8), 4e07f54 (R3/R4 transfer scaffold), final commit (complete calibration theorem; github.com/kivo360/plausibility).